

PROJECT REVIEW

RFP Sentinel

A buyer-side, multi-agent RAG platform for GeM Electronics procurement — checking tenders against government rules before publication, and bids against tender requirements after submission.

Objective

To build a software platform that helps a government buyer run a tender (RFP) fairly, quickly, and transparently — automatically checking a tender against real procurement rules before it is published, and automatically checking every submitted bid against that tender's own requirements once bidding closes.

The system is designed to assist the buyer's judgment, never replace it: every automated finding carries a citation back to the rule or evidence it is based on, and every consequential decision — publishing a flagged tender, resolving an ambiguous bid, breaking a price tie — still requires a human to confirm it.

Core positioning: most AI tools in the government-tender space are built for the company *bidding* on a tender. RFP Sentinel is built for the other side — the buyer / Technical Evaluator who has to check the tender is fair and then evaluate what comes back, honestly and defensibly.

Problem Statement

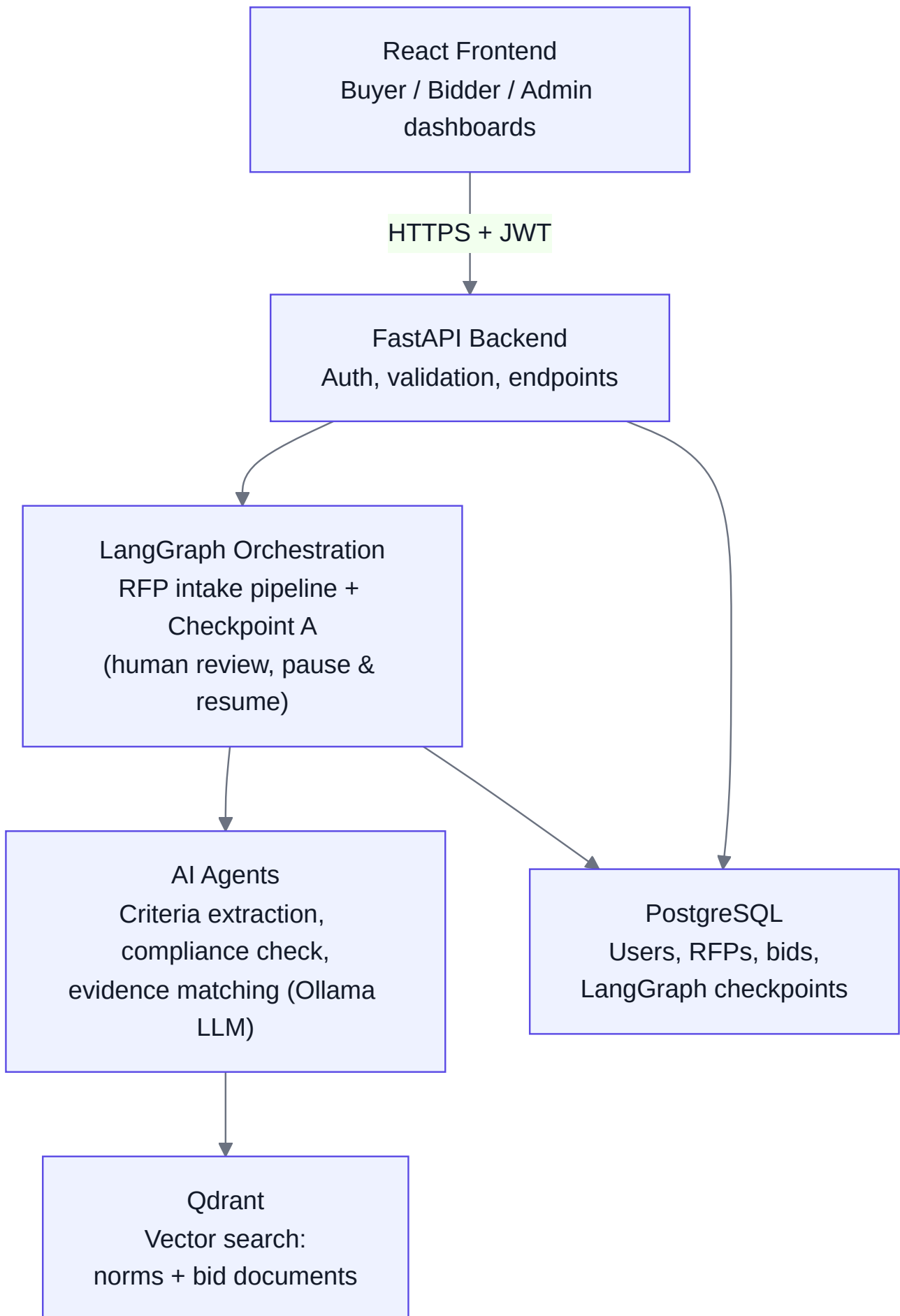
On India's Government e-Marketplace (GeM), a buyer department publishes a tender, companies submit bids against it, and someone has to fairly compare every bid against the rules and pick a winner. In practice, this process has three recurring problems:

- **Compliance mistakes go unnoticed until it's too late.** A tender clause that unintentionally conflicts with a government procurement rule (an MSME policy, a Make-in-India requirement, GeM's own terms) is often only caught after publication — once a bidder complains or the tender is legally challenged.
- **Bid evaluation is manual, slow, and inconsistent.** Reading through every bidder's documents and checking them against every eligibility and technical requirement, by hand, for each bid, does not scale and is easy to get wrong.
- **There is no defensible audit trail.** When a decision is questioned later, "we reviewed it and it seemed fine" is not a strong answer — a real citation to the specific rule or evidence is.

Existing AI tools in this space are built to help a *bidder* write a stronger proposal. Almost none address the buyer's side of the problem — checking the tender itself, and fairly evaluating what comes back.

Architecture

The system has four layers, each with a narrow responsibility. Requests flow downward from the frontend; only the layer that owns a piece of data ever touches its store directly.



- **React Frontend** — three role-based dashboards (Buyer, Bidder, Admin), talking to the backend over authenticated HTTPS calls.
- **FastAPI Backend** — validates every request, checks the user's role and session status, and either answers directly from the database or hands off slow, AI-heavy work to the orchestration layer.
- **LangGraph Orchestration** — runs the RFP intake pipeline as a sequence of steps, and is able to pause that sequence at a human review point (Checkpoint A) for as long as needed — even across a server restart — then resume exactly where it left off.
- **AI Agents (via Ollama)** — a small number of focused, LLM-backed steps: extracting structured criteria from an uploaded tender, checking those criteria against government rules, and checking a submitted bid's evidence against approved criteria. Every verdict must be backed by a real citation, or it is treated as inconclusive rather than trusted.
- **Qdrant + PostgreSQL** — Qdrant stores and searches document content by meaning (government norms, bid documents); PostgreSQL stores everything relational (accounts, tenders, bids) plus the orchestration layer's own saved progress.

Solution

RFP Sentinel implements this as two phases, each ending in a human decision point:

Phase 1 — Tender intake

- Buyer uploads the tender PDF; the system extracts its eligibility and technical requirements automatically.
- Every requirement is checked against a government-norms knowledge base and against GeM's own list of prohibited buyer practices.
- **Checkpoint A:** the buyer reviews any flagged requirement, edits it, or overrides it with a recorded reason, before publishing.

Phase 2 — Bid evaluation

- Once bidding closes, every bid's technical documents are checked against the buyer-approved requirements and scored.
- A bid failing any mandatory requirement is disqualified; one with inconclusive evidence is held for the buyer's own review, never guessed at automatically.
- Once the buyer opens financial bids, each qualifying bid's sealed price is read and the field is ranked — by lowest price, or by a blended quality-and-cost score, depending on the tender's own evaluation method.

What makes it defensible

- Every automated flag or verdict carries a citation to the rule or document it came from.
- The final scoring and ranking arithmetic is plain, deterministic code — not another AI judgment — so the same evidence always produces the same, explainable result.
- A bidder's price is never even read by the system until technical evaluation is fully complete, mirroring how real two-envelope government tenders are legally required to work.

Role of Every Team Member

Member	Area of ownership	Key contributions
Member 1	Backend & API Development	FastAPI application structure, authentication (JWT, roles, session handling), and all REST endpoints connecting the frontend to the rest of the system.
Member 2	AI / LLM Pipeline	Designed and built the LLM-based agents — criteria extraction, compliance checking, and bid evidence matching — including prompt design and citation-grounding logic.
Member 3	Orchestration & Workflow	Built the LangGraph pipeline connecting the AI steps together, including the human-in-the-loop checkpoint and its pause/resume behavior.
Member 4	Frontend & UI/UX	Built the React dashboards for all three roles (Buyer, Bidder, Admin) and the end-to-end user experience across the platform.
Member 5	Data & Infrastructure	Designed the PostgreSQL schema and Qdrant vector collections, document ingestion pipeline, and Docker-based local deployment.
Member 6	Testing, Scoring Logic & Documentation	Built the deterministic scoring engine and its test suite, and led testing, validation, and project documentation.

Note: names and exact role boundaries above are placeholders — replace with your team's actual members and contributions.